

Test Cases — main.dart Bugfixes

Testdokumentation für Flutter App (main.dart)

Datum: 02.06.2026

Tester: Jonathan Schmidt

Version: 1.0

Testumgebung: Flutter Web & Android

TC-001: Driving Mode — Stop-Befehl Timing

Kritisch

Funktional

Äquivalenzklasse: Verzögerte Befehlsausführung (Eingabemethoden / Tastatureingaben)

Befehlsverarbeitung

BESCHREIBUNG

Test der Stop-Befehl-Verzögerung beim Loslassen von Steuertasten.

VORBEDINGUNGEN

- App gestartet
- Driving Mode aktiv
- WebSocket-Verbindung zum Auto hergestellt

TESTSCHRITTE

1. Taste "W" (Vorwärts) drücken und 2 Sekunden halten
2. Taste loslassen
3. Zeit bis zum Stop-Befehl messen
4. Vorgang mit "A" (Links) wiederholen

ERWARTETES ERGEBNIS

- Auto fährt während Taste gehalten wird
- Stop-Befehl wird **500ms** nach Loslassen gesendet
- Auto stoppt nicht sofort beim Loslassen

TATSÄCHLICHES ERGEBNIS (VOR FIX)

FEHLGESCHLAGEN

- Stop-Befehl wurde nach nur **10ms** gesendet
- Auto stoppte unmittelbar nach jedem Tastendruck
- Kontinuierliches Fahren nicht möglich

TATSÄCHLICHES ERGEBNIS (NACH FIX)

BESTANDEN

- Stop-Befehl wird nach 500ms gesendet
- Guard-Bedingung prüft ob Taste noch gehalten wird
- Kontinuierliches Fahren funktioniert einwandfrei

ROOT CAUSE

```
// Bug: Timer zu kurz, keine Guard-Bedingung
_commandTimer = Timer(const Duration(milliseconds: 10), () {
  activeCommands.clear();
  updateCommand();
});
```

FIX

```
// Fix: 500ms Timer mit Guard
_commandTimer = Timer(const Duration(milliseconds: 500), () {
  if (!keyPressed.values.any((pressed) => pressed)) {
    if (activeCommands.isNotEmpty) {
      activeCommands.clear();
      updateCommand();
    }
  }
});
```

TC-002: Driving Mode — Stop-Befehl beim Button-Release

Kritisch**Funktional****Äquivalenzklasse: Fehlende Befehlsausführung (Eingabemethoden / Touch-Eingaben)****Befehlsausführung**

BESCHREIBUNG

Test ob Stop-Befehle beim Loslassen von UI-Buttons korrekt gesendet werden.

VORBEDINGUNGEN

- App gestartet
- Driving Mode aktiv
- Touch-Steuerung verfügbar

TESTSCHRITTE

1. Gas-Button antippen und halten (2 Sekunden)
2. Button loslassen
3. Beobachten ob Auto stoppt
4. Mit Lenk-Buttons (Links/Rechts) wiederholen
5. Mit Brems-Button wiederholen

ERWARTETES ERGEBNIS

- Auto reagiert während Button gedrückt
- Stop-Befehl wird beim Loslassen gesendet
- Auto stoppt nach Release

TATSÄCHLICHES ERGEBNIS (VOR FIX)

FEHLGESCHLAGEN

- activeCommands wurde aktualisiert
- updateCommand() wurde **nicht** aufgerufen
- Kein Stop-Befehl ans Auto gesendet
- Auto fuhr weiter

TATSÄCHLICHES ERGEBNIS (NACH FIX)

BESTANDEN

- updateCommand() wird in allen Release-Pfaden aufgerufen
- Stop-Befehl wird zuverlässig gesendet
- Auto stoppt korrekt

BETROFFENE FUNKTIONEN

- _pressAccelerate()
- _pressBrake()
- _pressSteerLeft()
- _pressSteerRight()

FIX

```
// Vorher: updateCommand() fehlte  
setState(() { accelerating = false; });  
  
// Nachher: updateCommand() hinzugefügt  
setState(() { accelerating = false; });  
updateCommand(); // NEU
```

TC-003: Driving Mode — Watchdog nur für Tastatur

Mittel**Funktional****Äquivalenzklasse: Sofortige Befehlsausführung (Eingabemethoden / Tastatureingaben)**

Befehlsausführung

BESCHREIBUNG

Test dass Watchdog-Timer nur bei Tastatur-Eingaben aktiviert wird, nicht bei Touch-Buttons.

VORBEDINGUNGEN

- App im Browser geöffnet
- Driving Mode aktiv

TESTSCHRITTE

1. Taste "W" drücken und halten
2. Browser-Tab wechseln (Fokus verlieren)
3. Zurück zum App-Tab wechseln
4. Prüfen ob Watchdog Stop-Befehl sendet

5. Gas-Button drücken und halten
6. Prüfen ob Watchdog aktiviert wird

ERWARTETES ERGEBNIS

- Watchdog aktiviert sich bei Tastatur-Eingaben
- Watchdog aktiviert sich **nicht** bei Touch-Buttons
- Stop-Befehl nach 500ms wenn kein Key mehr gehalten

TATSÄCHLICHES ERGEBNIS

BESTANDEN

- `_startCommandTimer()` nur in `_handleDrivingKey()` aufgerufen
- Touch-Buttons verwenden eigene Release-Handler
- Sicherheitsnetz für Browser KeyUp-Event-Verlust funktioniert

IMPLEMENTIERUNG

```
void _handleDrivingKey(RawKeyEvent event) {
  // ... Key-Handling ...
  if (isDown) {
    _startCommandTimer(); // Nur hier, nicht bei Buttons
  }
}
```

TC-004: Driving Mode — Button Long-Press Handling

Hoch

UI/UX

Äquivalenzklasse: Langes Halten (Nutzerinteraktionen / Touch-Eingaben)

Nutzerinteraktionen

BESCHREIBUNG

Test der Button-Reaktion bei langem Halten (Long-Press).

VORBEDINGUNGEN

- App gestartet
- Driving Mode aktiv

TESTSCHRITTE

1. Gas-Button antippen und 5 Sekunden halten
2. Button loslassen
3. Prüfen ob `onPointerUp` Event gefeuert wurde
4. Mit allen Steuer-Buttons wiederholen

ERWARTETES ERGEBNIS

- Button reagiert auf Press
- Button reagiert auf Release (auch nach Long-Press)

- Stop-Befehl wird immer gesendet

TATSÄCHLICHES ERGEBNIS (VOR FIX)

FEHLGESCHLAGEN

- GestureDetector.onTapUp wurde bei Long-Press unterdrückt
- Flutter klassifizierte Geste als Long-Press
- Button blieb "gedrückt"
- Stop-Befehl kam nie

TATSÄCHLICHES ERGEBNIS (NACH FIX)

BESTANDEN

- Listener.onPointerUp feuert bedingungslos
- Unabhängig von Gesten-Klassifizierung
- Stop-Befehl wird immer gesendet

FIX

```
// Vorher: GestureDetector
GestureDetector(
  behavior: HitTestBehavior.opaque,
  onTapDown: (_) => onHold(true),
  onTapUp: (_) => onHold(false),    // Wird bei Long-Press unterdrückt
  onTapCancel: () => onHold(false),
)

// Nachher: Listener
Listener(
  behavior: HitTestBehavior.opaque,
  onPointerDown: (_) => onHold(true),
  onPointerUp: (_) => onHold(false),    // Feuert immer
  onPointerCancel: (_) => onHold(false),
)
```

TC-005: Draw Route — Kurven-Klassifizierung

Hoch

Algorithmus

Äquivalenzklasse: Scharfe Kurven (Routenverarbeitung / Große Richtungsänderungen)

Routenverarbeitung

BESCHREIBUNG

Test der korrekten Erkennung von Links-/Rechtskurven und geraden Segmenten.

VORBEDINGUNGEN

- App gestartet
- Drawing Mode aktiv
- Zeichenfläche bereit

TESTSCHRITTE

1. Gerade Linie zeichnen (horizontal, 100px)
2. 90° Rechtskurve zeichnen
3. 90° Linkskurve zeichnen
4. Leichte Rechtskurve zeichnen (15°)
5. Playback starten und Befehle beobachten

ERWARTETES ERGEBNIS

- Gerade: `SegmentType.straight`
- 90° rechts: `SegmentType.sharpRight`
- 90° links: `SegmentType.sharpLeft`
- 15° rechts: `SegmentType.rightCurve`

TATSÄCHLICHES ERGEBNIS (VOR FIX)

FEHLGESCHLAGEN

- Vorzeichen-Konvention invertiert
- Links/Rechts vertauscht
- Gerade-Schwelle zu groß (10°)
- Viele Kurven als "gerade" klassifiziert

TATSÄCHLICHES ERGEBNIS (NACH FIX)

BESTANDEN

- Positiver Winkel = Rechtskurve
- Negativer Winkel = Linkskurve
- Gerade-Schwelle: $< 3^\circ$
- Sharp-Schwelle: $> 20^\circ$

SCHWELLWERTE

Parameter	Vorher	Nachher
Gerade	$< 10^\circ$	$< 3^\circ$
Sharp	$> 25^\circ$	$> 20^\circ$
Vorzeichen	invertiert	korrekt

TC-006: Draw Route — Command Timing

Hoch**Performance****Äquivalenzklasse: Leichte Kurven / Scharfe Kurven (Routenverarbeitung / Kleine & Große Winkeländerungen)**

Routenverarbeitung

BESCHREIBUNG

Test der Command-Dauer-Berechnung für verschiedene Segment-Typen.

VORBEDINGUNGEN

- Route gezeichnet mit verschiedenen Segment-Typen
- Playback bereit

TESTSCHRITTE

1. Route mit 30px geradem Segment zeichnen
2. Route mit 30px Kurve zeichnen
3. Route mit 30px scharfer Kurve zeichnen
4. Playback starten
5. Command-Dauern messen

ERWARTETES ERGEBNIS

- Minimum-Dauer: 80ms
- Kurven-Split: 90% Lenken / 10% Fahren
- Sharp-Multiplikator: 5x

TATSÄCHLICHES ERGEBNIS (VOR FIX)

FEHLGESCHLAGEN

- Minimum-Dauer: 100ms (zu lang)
- Kurven-Split: 70% / 30% (zu wenig Lenkzeit)
- Sharp-Multiplikator: 3x (zu wenig für 90° Kurven)
- Auto überfuhr Kurven

TATSÄCHLICHES ERGEBNIS (NACH FIX)

BESTANDEN

- Minimum-Dauer: 80ms
- Kurven-Split: 90% / 10%
- Sharp-Multiplikator: 5x
- Auto navigiert Kurven präzise

PARAMETER-ÄNDERUNGEN

Parameter	Vorher	Nachher
Minimum-Dauer	100 ms	80 ms
Kurven-Split	70% / 30%	90% / 10%
Sharp-Multiplikator	3x	5x

TC-007: Draw Route — Segment-Mindestabstand

Kritisch

Performance

Äquivalenzklasse: Kurze Segmente (Algorithmische Grenzfälle / Minimale Streckenlänge)

Algorithmische Grenzfälle

BESCHREIBUNG

Test der Segment-Generierung und Anzahl bei verschiedenen Mindestabständen.

VORBEDINGUNGEN

- Drawing Mode aktiv
- Leere Zeichenfläche

TESTSCHRITTE

1. Komplexe Route zeichnen (ca. 500px Gesamtlänge)
2. Segment-Anzahl zählen
3. Command-Anzahl zählen
4. Playback-Dauer messen
5. Timer-Drift beobachten

ERWARTETES ERGEBNIS

- 10-15 Segmente pro Route
- ~15-20 Commands
- Playback-Dauer: 5-10 Sekunden
- Minimaler Timer-Drift

TATSÄCHLICHES ERGEBNIS (VOR FIX)

FEHLGESCHLAGEN

- Mindestabstand: 5px
- ~100+ Segmente pro Route
- ~120 Befehle generiert
- Playback-Dauer: 36+ Sekunden
- Akkumulierter Timer-Drift
- Auto kam nicht am Ziel an

TATSÄCHLICHES ERGEBNIS (NACH FIX)

BESTANDEN

- Mindestabstand: 30px
- 10-15 Segmente pro Route
- ~15-20 Befehle
- Playback-Dauer: 5-10 Sekunden
- Minimaler Drift

METRIKEN

--	--	--

Metrik	Vorher (5px)	Nachher (30px)
Segmente	100+	10-15
Commands	~120	~15-20
Dauer	36+ sec	5-10 sec
Drift	Hoch	Minimal

TC-008: Draw Route — Punkt-Glättung

Mittel**Algorithmus****Äquivalenzklasse: Kleine Winkel (Algorithmische Grenzfälle / Kaum erkennbare Richtungsänderung)****Algorithmische Grenzfälle**

BESCHREIBUNG

Test der Punkt-Glättung für saubere Kurven-Erkennung.

VORBEDINGUNGEN

- Drawing Mode aktiv
- Maus/Touch-Eingabe verfügbar

TESTSCHRITTE

1. Schnell eine wellige Linie zeichnen (Hand-Zittern simulieren)
2. Segment-Klassifizierung beobachten
3. Anzahl der Kurven-Segmente zählen
4. Playback starten und Fahrt beobachten

ERWARTETES ERGEBNIS

- Glatte Kurven trotz zittriger Eingabe
- Wenige, saubere Segmente
- Flüssige Fahrt

TATSÄCHLICHES ERGEBNIS (VOR FIX)

FEHLGESCHLAGEN

- 3-Punkt-Glättung unzureichend
- Viele kleine Zick-Zack-Segmente
- Ruckelige Fahrt
- Kurven-Erkennung ungenau

TATSÄCHLICHES ERGEBNIS (NACH FIX)

BESTANDEN

- 5-Punkt gleitendes Mittel
- Glatte Segmente
- Flüssige Fahrt
- Präzise Kurven-Erkennung

ALGORITHMUS

```
// Vorher: 3-Punkt-Glättung
smoothed[i] = (points[i-1] + points[i] + points[i+1]) / 3;

// Nachher: 5-Punkt-Glättung
smoothed[i] = (points[i-2] + points[i-1] + points[i] +
               points[i+1] + points[i+2]) / 5;
```

TC-009: Draw Route — Scale-Faktor für Vollgas-Motor

Hoch

Kalibrierung

Äquivalenzklasse: Kurze Segmente (Algorithmische Grenzfälle / Minimale Streckenlänge)

Algorithmische Grenzfälle

BESCHREIBUNG

Test der Distanz-zu-Zeit-Konvertierung für Vollgas-Motor.

VORBEDINGUNGEN

- Auto mit Vollgas-Motor (On/Off, keine Geschwindigkeitsregelung)
- Route gezeichnet

TESTSCHRITTE

1. 30px gerades Segment zeichnen
2. Speed-Multiplier auf 1.0x setzen
3. Erwartete Dauer berechnen
4. Playback starten
5. Tatsächliche zurückgelegte Strecke messen
6. Mit 0.5x wiederholen

ERWARTETES ERGEBNIS

- 30px bei 1.0x: ~120ms
- 30px bei 0.5x: ~240ms
- Auto fährt korrekte Distanz

TATSÄCHLICHES ERGEBNIS (VOR FIX)

FEHLGESCHLAGEN

- Scale: x 10

- Minimum: 150ms
- 30px bei 1.0×: 300ms (zu lang)
- Auto überfuhr kurze Strecken deutlich

TATSÄCHLICHES ERGEBNIS (NACH FIX)

BESTANDEN

- Scale: × 4
- Minimum: 80ms
- 30px bei 1.0×: 120ms
- Präzise Distanz-Kontrolle

KALIBRIERUNG

Setting	Segment (30px)	Vorher	Nachher
1.0×	Gerade	300 ms	120 ms
0.5×	Gerade	600 ms	240 ms

TC-010: Draw Route — Kurven-Segment-Merge

Hoch**Algorithmus****Äquivalenzklasse: Scharfe Kurven (Routenverarbeitung / Große Richtungsänderungen)****Routenverarbeitung**

BESCHREIBUNG

Test des Zusammenführens aufeinanderfolgender Kurven-Segmente.

VORBEDINGUNGEN

- Drawing Mode aktiv
- Route mit 90° Ecke

TESTSCHRITTE

1. Route mit scharfer 90° Rechtskurve zeichnen
2. Segment-Analyse beobachten
3. Anzahl der Kurven-Segmente zählen
4. Anzahl der Lenk-Befehle zählen
5. Playback starten
6. Lenkverhalten beobachten

ERWARTETES ERGEBNIS

- Eine 90° Ecke = 1 Kurven-Segment
- 1 Lenk-Befehl pro Ecke
- Auto lenkt einmal und fährt weiter

TATSÄCHLICHES ERGEBNIS (VOR FIX)**FEHLGESCHLAGEN**

- Eine 90° Ecke → 2-3 Kurven-Segmente (nach Glättung)
- 2-3 separate Lenk-Befehle
- Auto überdrehte
- Zu starke Lenkung

TATSÄCHLICHES ERGEBNIS (NACH FIX)**BESTANDEN**

- `_mergeConsecutiveCurves()` implementiert
- Aufeinanderfolgende gleich-gerichtete Kurven zusammengefasst
- 1 Segment pro Ecke
- 1 Lenk-Befehl pro Ecke
- Präzise Lenkung

ALGORITHMUS

```
List<RouteSegment> _mergeConsecutiveCurves(List<RouteSegment> segments) {
  // Fasst aufeinanderfolgende Kurven gleicher Richtung zusammen
  // leftCurve + leftCurve → 1× leftCurve
  // rightCurve + rightCurve → 1× rightCurve
}
```

TC-011: Draw Route — State Cleanup

Mittel

State Management

Äquivalenzklasse: Inkonsistenter Zustand (Zustandsmanagement / Veraltete Befehle sind aktiv)

Zustandsmanagement

BESCHREIBUNG

Test der Zustandsbereinigung beim Wechsel von Driving zu Drawing Mode.

VORBEDINGUNGEN

- Driving Mode aktiv
- Mehrere Tasten/Buttons gedrückt

TESTSCHRITTE

1. Im Driving Mode: "W" + "A" gleichzeitig drücken und halten
2. Zu Drawing Mode wechseln
3. Route zeichnen
4. Playback starten
5. Gesendete Befehle beobachten

ERWARTETES ERGEBNIS

- Nur Route-Befehle werden gesendet
- Keine Driving-Mode-Befehle
- Sauberer State

TATSÄCHLICHES ERGEBNIS (VOR FIX)

FEHLGESCHLAGEN

- `activeCommands` enthielt: `forward`, `left`
- `keyPressed` enthielt: `W`, `A`
- Playback sendete kombinierte Befehle: `forward`, `left`, `right`
- Auto fuhr unkontrolliert

TATSÄCHLICHES ERGEBNIS (NACH FIX)

BESTANDEN

- `activeCommands.clear()` am Anfang von `_startPlayback()`
- `keyPressed.clear()` am Anfang von `_startPlayback()`
- Nur Route-Befehle werden gesendet
- Saubere Fahrt

FIX

```
void _startPlayback() async {  
  activeCommands.clear(); // NEU  
  keyPressed.clear();      // NEU  
  // ... Rest der Playback-Logik  
}
```

TC-012: Draw Route — UI Cleanup

Niedrig**UI/UX****Äquivalenzklasse: Normale Nutzung (Systemverhalten und Performanceverhalten / Standardnutzung)****System- & Performanceverhalten**

BESCHREIBUNG

Test der UI-Bereinigung (entfernte nicht-funktionale Elemente).

VORBEDINGUNGEN

- Drawing Mode geöffnet

TESTSCHRITTE

1. UI inspizieren
2. Nach Geschwindigkeitsregler suchen
3. Nach Save-Button suchen

4. Code-Review: `_speedMultiplier` Deklaration prüfen

ERWARTETES ERGEBNIS

- Kein Geschwindigkeitsregler sichtbar
- Kein Save-Button sichtbar
- `_speedMultiplier` als `final` deklariert (Wert: 1.0)

TATSÄCHLICHES ERGEBNIS

BESTANDEN

- Geschwindigkeitsregler entfernt (macht bei Vollgas-Motor keinen Sinn)
- Save-Button entfernt (keine Speicherfunktion vorhanden)
- `final double _speedMultiplier = 1.0;`
- Saubere, fokussierte UI

Test-Zusammenfassung

STATISTIK

- **Gesamt Tests:** 12
- **Kritisch:** 3
- **Hoch:** 5
- **Mittel:** 3
- **Niedrig:** 1

ERGEBNISSE (VOR FIXES)

- **Fehlgeschlagen:** 11
- **Bestanden:** 1

ERGEBNISSE (NACH FIXES)

- **Bestanden:** 12
- **Fehlgeschlagen:** 0

KATEGORIEN

- Funktional: 6 Tests
- UI/UX: 3 Tests
- Algorithmus: 4 Tests
- Performance: 2 Tests
- State Management: 1 Test

KRITISCHE BUGS BEHOBEN

1. Stop-Befehl Timing (TC-001)
2. Fehlende `updateCommand()` Aufrufe (TC-002)
3. Segment-Mindestabstand Performance (TC-007)

Lessons Learned

1. **Timer-Werte kritisch:** 10ms vs 500ms macht den Unterschied zwischen funktionierend und nicht-funktionierend
2. **Flutter Gesture System:** GestureDetector unterdrückt Events bei Long-Press → Listener verwenden
3. **State Management:** Globale States müssen beim Mode-Wechsel bereinigt werden
4. **Algorithmus-Kalibrierung:** Vollgas-Motor benötigt andere Parameter als Geschwindigkeits-geregelte Motoren
5. **Glättung wichtig:** 5-Punkt-Glättung deutlich besser als 3-Punkt für Kurven-Erkennung

Dokumentiert von: Alexa van der Meulen | **Review:** Team R.E.D. | **Status:** Alle Tests bestanden